SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE:CSA1703

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.* (BTL 5)

Assessment Tool Weightage: 50%

Duration : 1 Hour
QUESTIONS: 5 **Total Marks:50**

Annexure A

Design Task

Code of Conduct:

*I **K. Pujitha (192524149**) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K. Pujitha

Annexure A

Design Task

1. **Hybrid AI Recommendation Engine Design**

   Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.

2. **Smart Disaster Detection System**

Develop a distributed AI system using PySpark and RDD operations to detect natural disasters (such as floods or earthquakes) from multi-source data streams (satellite images, sensors, and social media). Describe the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.

3. **AI-Based Fraud Detection Framework**

Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.

4. **Autonomous Supply Chain Optimization System**

Construct a distributed AI system using PySpark for optimizing supply chain operations (inventory, logistics, demand forecasting). Describe how RDD transformations support large-scale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.

5. **Personalized Learning Analytics Platform**

Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined

Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation

1. Hybrid AI Recommendation Engine

Problem

A recommendation system must suggest relevant products, movies, courses, etc. to users. A single recommendation technique may not provide accurate results.

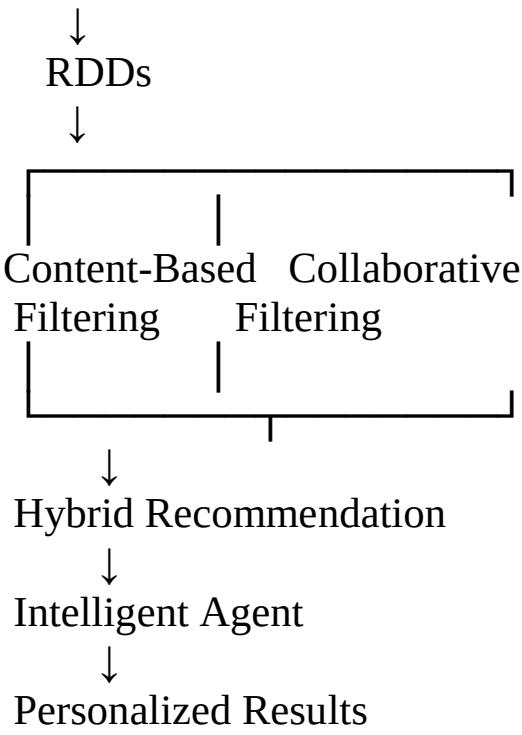
Objective

Combine:

- **Content-Based Filtering** → recommends items similar to what the user already likes.
- **Collaborative Filtering** → recommends items liked by similar users.

Architecture

User-Item Data
↓
PySpark



RDD operations

```
ratings = sc.parallelize([
    ("U1", "Item1", 5),
    ("U1", "Item2", 3),
    ("U2", "Item1", 4)
])

high_ratings = ratings.filter(lambda x: x[2] >= 4)

user_items = high_ratings.map(
    lambda x: (x[0], x[1])
)

result = user_items.collect()
print(result)
```

Intelligent Agents

- User Profile Agent → monitors user interests.

- **Recommendation Agent** → generates recommendations.
- **Feedback Agent** → observes likes/dislikes/clicks.
- **Adaptation Agent** → updates recommendations based on new behavior.

Scalability

PySpark divides millions of user-item records among multiple worker nodes, allowing processing in parallel.

Innovation

The system continuously changes recommendations according to real-time user behavior.

2. Smart Disaster Detection System

Problem

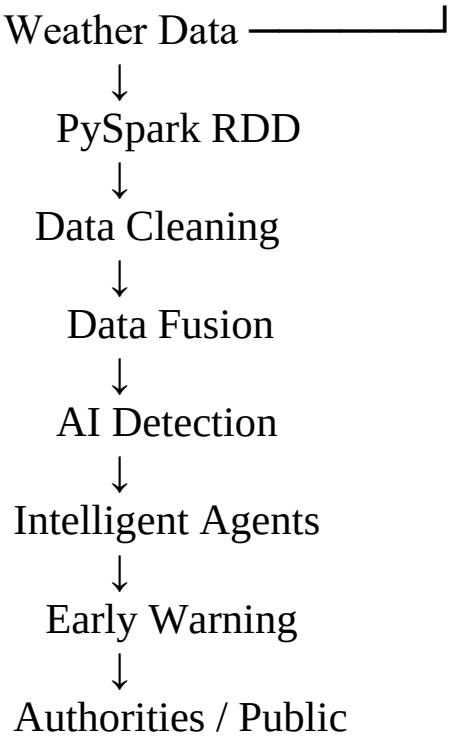
Natural disasters need to be detected quickly using information from many sources.

Objective

Detect floods, earthquakes, etc. by combining:

- Satellite images
- Sensors
- Social media
- Weather data

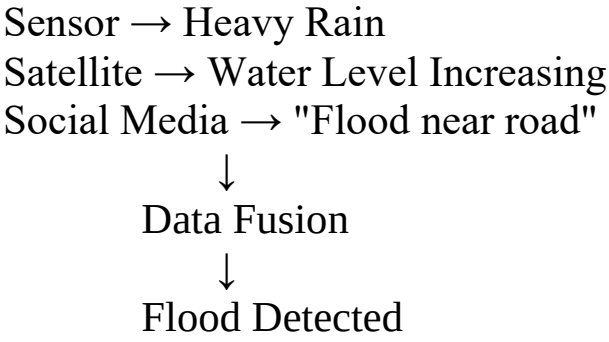
Architecture



Data Fusion

Different sources are combined to improve confidence.

Example:



RDD operations

```
data = sc.parallelize([
    ("sensor", "high_water"),
    ("satellite", "flood_area"),
    ("social", "flood_report")
```

])

```
alerts = data.filter(lambda x: "flood" in x[1])
print(alerts.collect())
```

Intelligent Agents

- **Sensor Agent** → monitors sensor readings.
- **Satellite Agent** → analyzes satellite information.
- **Social Media Agent** → identifies disaster-related posts.
- **Warning Agent** → sends alerts.
- **Response Agent** → recommends evacuation/rescue actions.

Innovation

Multiple agents cooperate to detect disasters earlier than relying on a single source.

3. AI-Based Fraud Detection Framework

Problem

Banks and financial systems process millions of transactions. Fraudulent transactions may be hidden among normal transactions.

Objective

Detect suspicious transactions using distributed anomaly detection.

Architecture

Financial Transactions
↓
PySpark
↓

RDD
↓
Data Cleaning & Filtering
↓
Feature Extraction
↓
Anomaly Detection
↓
Intelligent Agents
↓
Fraud Score
↓
Alert / Block Transaction

Example

```
transactions = sc.parallelize([
   ("T1", 500),
   ("T2", 700),
   ("T3", 50000),
   ("T4", 300)
])

suspicious = transactions.filter(
   lambda x: x[1] > 10000
)

print(suspicious.collect())
```

Here, transactions above ₹10,000 are treated as suspicious only for demonstration.

Intelligent Agents

- **Transaction Agent** → monitors transactions.
- **Anomaly Agent** → identifies unusual behavior.

- **Risk Agent** → calculates fraud risk.
- **Alert Agent** → generates alerts.
- **Learning Agent** → learns from confirmed fraud cases.

Scalability

Transactions are distributed across Spark workers, allowing millions of records to be processed simultaneously.

Innovation

The agents continuously learn from new fraud patterns and improve detection accuracy.

4. Autonomous Supply Chain Optimization

Problem

Companies need to manage:

- Inventory
- Transportation
- Demand
- Warehouses
- Delivery

Poor decisions can increase cost and delays.

Objective

Use distributed AI to predict demand and optimize inventory and logistics.

Architecture

Sales Data

Inventory Data
Transport Data
Demand Data
↓
PySpark
↓
RDD
↓
Data Processing
↓
Demand Forecasting
↓
Optimization
↓
Intelligent Agents
↓
Inventory + Logistics Decisions
↓
Cost Reduction

RDD example

```
sales = sc.parallelize([
   ("ProductA", 100),
   ("ProductB", 50),
   ("ProductA", 150),
   ("ProductB", 80)
])

total_sales = sales.reduceByKey(
   lambda a, b: a + b
)

print(total_sales.collect())
```

Output:

[('ProductA', 250), ('ProductB', 130)]

Intelligent Agents

- **Inventory Agent** → checks stock levels.
- **Demand Agent** → predicts future demand.
- **Logistics Agent** → selects efficient routes.
- **Warehouse Agent** → manages warehouse operations.
- **Cost Agent** → tries to minimize total cost.

Innovation

Agents can make decentralized decisions. For example, the inventory agent can request stock from a warehouse when demand increases.

5. Personalized Learning Analytics Platform

Problem

Students have different learning speeds, strengths and weaknesses. Giving the same content to everyone is not always effective.

Objective

Analyze student behavior and performance to provide personalized learning.

Architecture

Student Data
↓
Quiz / Exam Scores
Learning Time
Clicks / Activities

↓
PySpark
↓
RDDs
↓
Learning Analytics
↓
Performance Prediction
↓
Intelligent Agents
↓
Personalized Content
↓
Real-Time Feedback

RDD example

```
students = sc.parallelize([
   ("A", 85),
   ("B", 45),
   ("C", 70),
   ("D", 35)
])

weak_students = students.filter(
   lambda x: x[1] < 50
)

print(weak_students.collect())
```

Output:

```
[('B', 45), ('D', 35)]
```

The system can recommend additional learning material to these students.